AI-Assisted Development

Realita, Strategi, dan Praktik
untuk Tim Produk & Bisnis Digital

👋 Perkenalan

https://www.zainfathoni.com/about

- 📍 Jember ➡️ Bandung ➡️ 🇸🇬 SG ➡️ Jogja
- 🛠️ Backend ➡️ Manager ➡️ Frontend ➡️ Fullstack
- 🤖 AI-Assisted Development Practitioner
- 🤝 Talk ini untuk semua: dev, founder, marketer, dan produk

Agenda

1. **The Reality of AI-Assisted Development**
2. **Coverage & Control**
3. **Prompting & Interaction Strategy**
4. **Structured AI Coding**
5. **Maintaining Craftsmanship**
6. **Live Demo Results** 👀

🎬 Live Demo Setup

Prompt AI di Awal, Cek Hasil di Akhir

```
"Buatkan slide presentasi untuk talk saya tentang
 AI-Assisted Development, lihat format slide yang
 sudah ada di repository ini."
```

⏳ AI bekerja di background selama presentasi...

1) The Reality of AI-Assisted Development

🤖 Peran AI di Setiap Fase Kerja Digital

Fase	Tools AI	Peran
🎨 **Design**	v0, Figma AI	Prototyping UI dan landing page
📝 **Content**	ChatGPT, Claude	Draft copy, outline, dokumentasi
💻 **Coding**	Claude Code, Copilot	Implementasi
🧪 **Testing**	Playwright MCP	Pengujian otomatis
🔍 **Review**	AI Code Review	Deteksi masalah
🚀 **Deploy**	Vercel, CI/CD	Otomasi deployment

✨ Hype vs Reality

Hype:

> "AI will replace developers"

Realita:

- ✅ AI **mempercepat** pekerjaan repetitif
- ✅ AI **membantu** eksplorasi solusi
- ❌ AI **belum bisa** menggantikan pemahaman arsitektur
- ❌ AI **sering salah** di edge case dan konteks bisnis
- ❌ AI **butuh arahan** yang jelas dari manusia yang paham konteks

⚠️ Jebakan Umum

- 🙈 **Copy-paste tanpa review** → bug tersembunyi
- 🍝 **Tanpa arsitektur** → sulit di-maintain
- 🔓 **Security blind spots** → vulnerability lolos
- 💣 **Over-confidence** → "AI yang bikin, pasti benar"

👥 Siapa yang Ada di Ruangan Ini?

Snapshot Peserta EDC Jogja

- 🏢 **Mayoritas dari sektor TIK** (25 dari 49 peserta)
- 📍 **39 dari 49 berdomisili di DIY**
- 👩‍💻 Peran terbesar: **Software Developer + Web Developer** (14)
- 💼 Tapi ruangan ini juga berisi **founder, business owner, IT manager,**
  digital marketer, dan mahasiswa
- 🌱 Usia dominan: **22-35 tahun**

👉 **Fokus hari ini: AI untuk kolaborasi lintas fungsi, bukan cuma**

🗣️ Cerita Nyata: Pengalaman Pribadi

Yang Berhasil ✅

- Migrasi framework Remix → React Router v7 — AI handle 80% boilerplate, saya fokus di routing logic
- Refactor styling ke Tailwind — AI konsisten di seluruh 30+ komponen
- Draft copy, struktur presentasi, dan dokumentasi awal — AI bantu mulai lebih cepat

Yang Gagal ❌

- AI generate test yang pass tapi **tidak test apa-apa** (assertion

💡 Jadi Gimana?

- 🛡️ **Trust, but Verify** — selalu review
- 🧪 **Automate checks** — biarkan mesin bantu verifikasi
- 📋 **Mulai dari spec** — tujuan dulu, baru eksekusi
- 🧠 **Pahami output-nya** — jangan jadi passenger

2) Coverage & Control

📊 Coverage vs Control

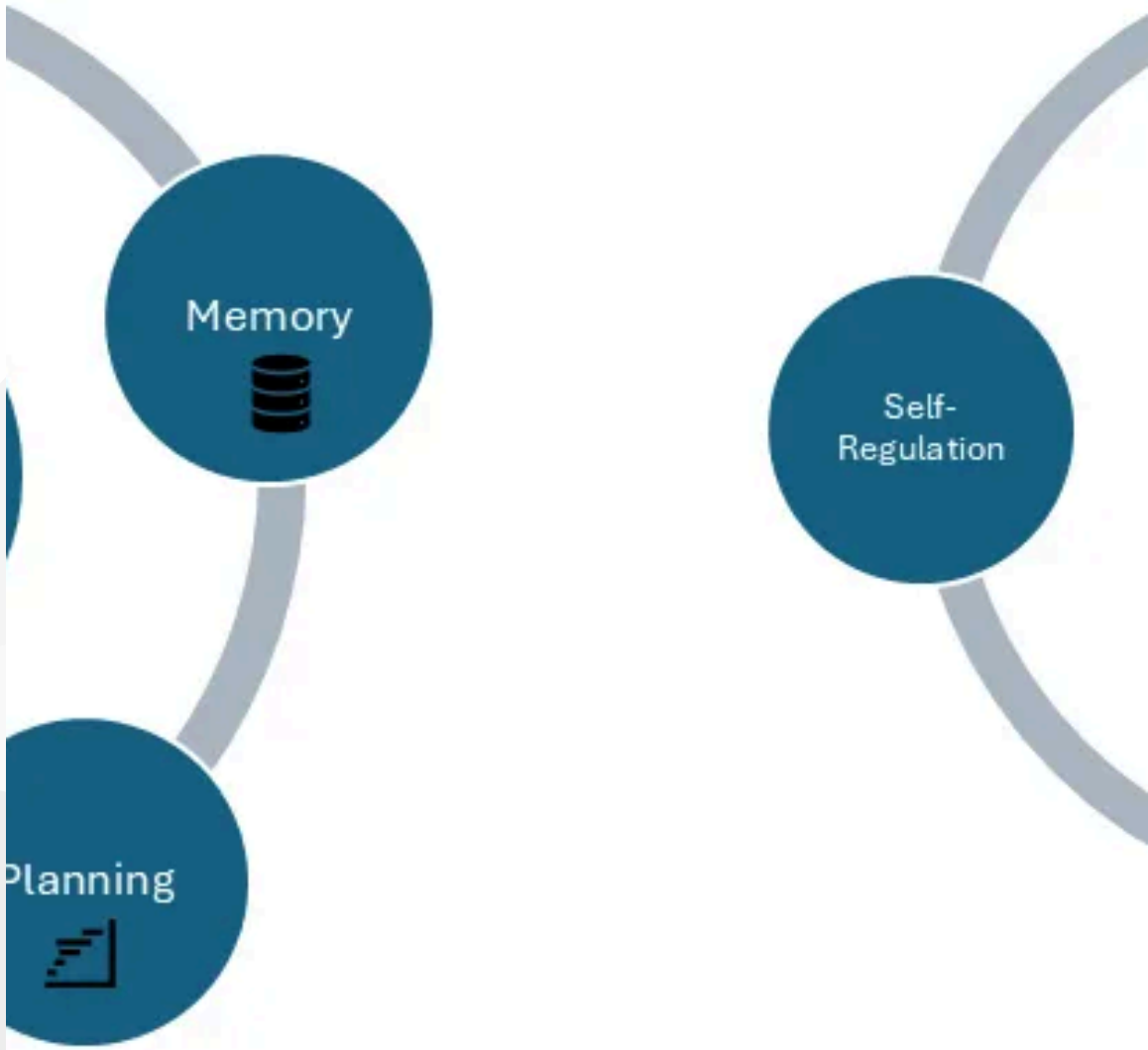
Coverage

Seberapa banyak AI dilibatkan dalam proses kerja

Control

Seberapa besar kendali manusia atas hasilnya

Keduanya harus seimbang



© CloudDon LLC

🚦 Level Kontrol Penggunaan AI

1. 🟢 Assistive — AI menyarankan, manusia memutuskan

Autocomplete, inline suggestions, grammar check

2. 🟡 Semi-Autonomous — AI kerjakan, manusia review

Draft copy, generate function, buat proposal awal

3. 🔴 Autonomous — AI end-to-end, manusia verifikasi akhir

🧭 Framework: Kapan & Seberapa Besar

Situasi	Coverage	Control	Contoh
🎨 Prototyping	⬆️ Tinggi	⬇️ Rendah	Bolt.new, v0
📢 Marketing/content	⬆️ Tinggi	↔️ Sedang	Outline, copy draft, content plan
💼 Business planning	↔️ Sedang	⬆️ Tinggi	Riset kompetitor, ringkasan meeting, draft proposal
🏗️ Production	↔️ Sedang	⬆️ Tinggi	Claude Code + Review

🔑 Kamu Tetap Pilot-nya

> "AI can accelerate development, but it cannot replace fundamental
> software engineering principles." — Addy Osmani

- 👨‍✈️ AI adalah **co-pilot**, bukan captain
- 🧠 Pahami keputusan pentingnya
- ✅ Yang bertanggung jawab tetap **kamu**

3) Prompting & Interaction Strategy

⌨️ Prompting 101

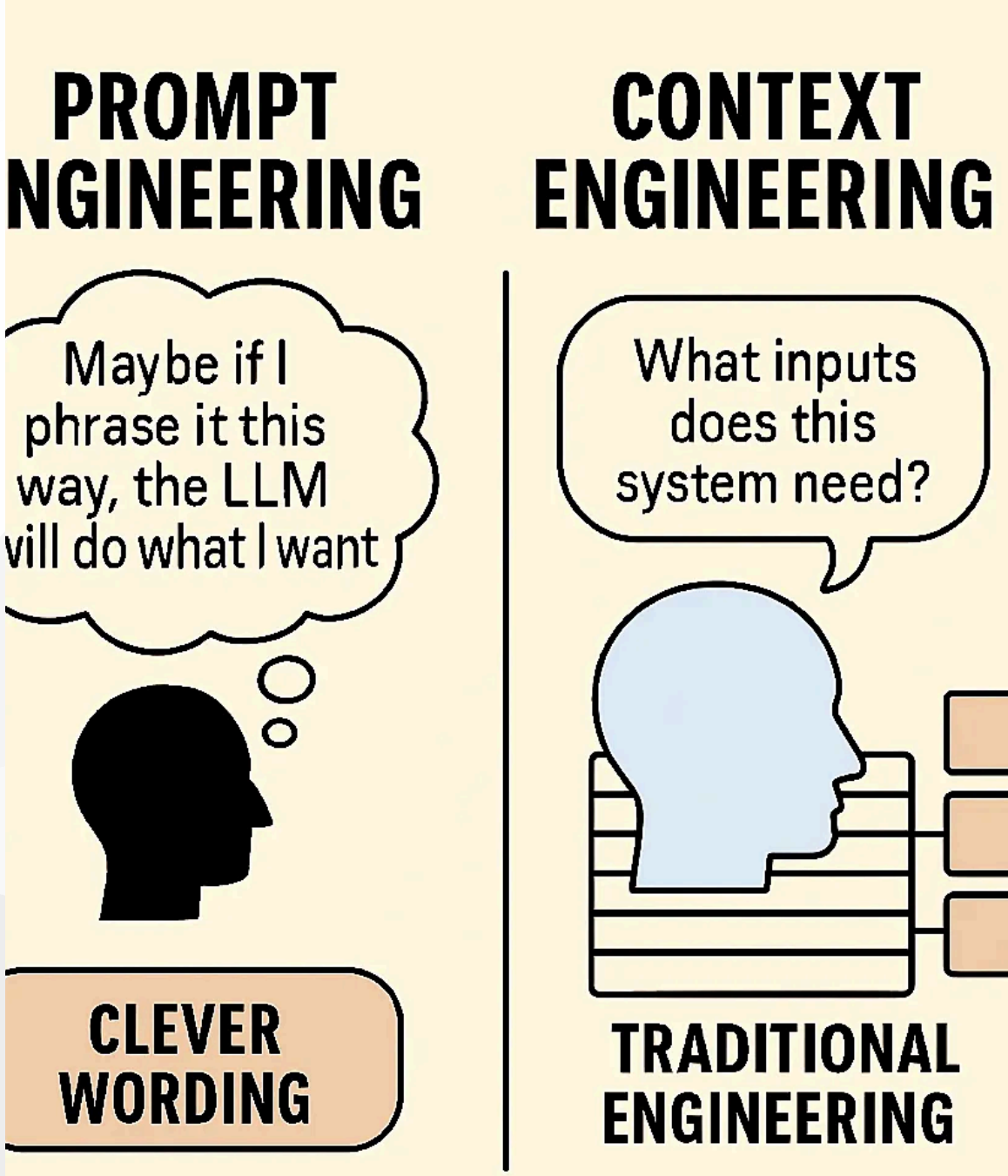
Prompt yang Baik = Hasil yang Baik

- 🎯 **Spesifik** — detail, bukan "buatkan sesuatu"
- 📋 **Kasih konteks** — file, constraint, target audiens
- 🖼️ **Kasih contoh** — tunjukkan format yang dimau
- 🔄 **Iterasi** — perbaiki berdasarkan hasil

🧠 Context Engineering > Prompt Engineering

Bukan hanya tentang **cara bertanya**,
tapi **informasi apa yang tersedia**

- ⌨️ **User Input** — prompt kamu
- 📚 **Knowledge** — docs, transcript, riset
- 💬 **Conversation** — konteks



👥 Role-Based Interaction

Gunakan AI Sesuai Perannya

🏗️ AI sebagai Architect

"Desain arsitektur untuk fitur X dengan constraint Y"

💻 AI sebagai Implementer

"Implementasikan function X sesuai spec ini"

📝 AI sebagai Writing Partner

"Buatkan draft landing page untuk audience UMKM teknologi di Jogja"

💼 AI sebagai Business Analyst

📂 AI sebagai Second Brain

Insight dari Lenny's Newsletter

- 🧠 **Context is the new interface**
- 📚 Bukan cuma prompt — docs, riset, feedback, keputusan
- 🔍 Context tersimpan rapi → bukan cuma ingat lebih banyak, tapi **berpikir lebih jernih**
- 👥 Berlaku untuk semua fungsi, bukan cuma developer

🔄 Iterasi, Bukan Sekali Jadi

1. 1️⃣ **Prompt awal** → draft pertama
2. 2️⃣ **Review** → identifikasi kekurangan
3. 3️⃣ **Refine** → tambah constraint/context
4. 4️⃣ **Ulangi** sampai kualitas sesuai

💡 **Tips:** Gunakan `plan mode` untuk hemat token

⚡ Prototype dalam Menit, Bukan Minggu

Insight dari Lenny's Newsletter

- 🚀 Prototype interaktif bisa jadi dalam **hitungan menit**
- 💬 Uji ide **sebelum commit build penuh**
- 🎨 Untuk **semua fungsi** — samakan persepsi sebelum eksekusi
- ⚠️ Mulai kecil, iterasi bertahap

👉 **Prototype = alat belajar tercepat**

📜 CLAUDE.md = Autopilot Rules

Konsistensi Output Otomatis

```
# CLAUDE.md
## Git Conventions
- Do not include co-authorship in commit messages

## Architecture
- React Router v7 with file-based routing
- Tailwind CSS for styling
- TypeScript for type safety
```

👉 AI membaca ini dan mengikuti konvensi project

4) Structured AI Coding

🪜 Jangan Langsung Coding!

1. 📋 **Audience & goal** — untuk siapa, dan apa hasil yang dibutuhkan?
2. 🔄 **Flow** — bagaimana user atau tim akan memakainya?
3. 🏗️ **Approach** — bagaimana cara terbaik mengerjakannya?
4. 💻 **Implementation** — baru mulai eksekusi

🔄 Tanpa vs Dengan Struktur

❌ Tanpa Struktur

```
"Buatkan fitur authentication untuk app saya"
→ AI generates random auth flow, may not match your stack
→ Missing edge cases, no error handling spec
```

✅ Dengan Struktur

```
Requirement: User login via email/password + OAuth
Flow: Login → Validate → Session → Redirect
Architecture: Server-side session, bcrypt, OAuth2
Constraints: Rate limiting, CSRF protection
→ AI delivers spec-compliant implementation
```

🧠 Prinsip Dasar

- ⚡ **Cepat tanpa struktur** = technical debt
- 🛡️ **Output tanpa review** = liability
- 📈 **Struktur + AI** = sustainable velocity

Mantra:

🎯 Define → 🧪 Test → 👀 Review

📝 Brief yang Terstruktur

Audience → Goal → Flow → Approach → Eksekusi

- 📝 **requirements** — kebutuhan yang bisa dicek
- 🎨 **design** — pendekatan dan alur kerja
- 📋 **tasks** — rencana yang trackable

👉 **AI bekerja lebih baik kalau brief-nya jelas**

5) Maintaining Craftsmanship

💎 Kualitas Tetap Nomor Satu

- 🔍 **Review setiap output** — jangan auto-accept
- 🧪 **Automate** — unit test, integration, E2E
- 📋 **Standar jelas** — checklist, style guide, acceptance criteria
- 👥 **Peer review** — mata manusia tetap penting

> "The 70% problem: AI gets you 70% of the way there remarkably fast, but the
> remaining 30% — the hard parts — still require real engineering skill."
> — Addy Osmani

📋 Prompt Saja Tidak Cukup → Butuh Evals

Insight dari Lenny's Newsletter

- 🧪 Untuk AI features, butuh **evaluasi terukur**
- 🔍 Cek: **correctness, hallucination, tone, relevance**
- 🔁 Evals bikin improvement **terukur**, bukan feeling
- ⚠️ **Vibe check ≠ quality check**

🔍 AI untuk Review

- 👃 **Code smell** — duplicated logic, god functions
- ⚠️ **Anti-pattern** — circular deps, tight coupling
- 🐌 **Inefficiency** — N+1 queries, unnecessary re-renders
- 🔒 **Security** — injection, XSS, exposed secrets

🔒 Security: Manusia = Final Gatekeeper

- 🛡️ Validasi **vulnerability** — injection, insecure API, deps
- ⛔ Jangan blindly trust AI untuk security
- ✅ **Checklist** sebelum deploy
- 🤖 AI boleh **scan**, tapi manusia yang **approve**

⚙️ System Thinking

- 🧩 **Konsistensi** — satu pattern, seluruh produk
- ⛔ Hindari **over-fragmentation**
- 🔗 **Pahami impact** setiap perubahan
- 📚 **Docs** — AI bantu generate, manusia review

✅ Do's

- 🔍 **Review** setiap output
- 📋 **Konteks lengkap** — CLAUDE.md, spec, constraint
- 🧪 **Test + evals** otomatis
- 🔄 **Iterasi** bertahap
- 🧠 **Pahami** apa yang di-generate
- 📜 **Rules file** untuk konsistensi

❌ Don'ts

- 🙈 **Copy-paste tanpa baca**
- ❓ **Prompt ambiguous** — garbage in, garbage out
- 👀 **Vibe check doang** — bukan evaluasi
- 💻 **"Works on my machine"** — test di CI
- 🔮 **Expect sempurna sekali jadi**
- 😴 **Jadi passenger** di output sendiri
- 🚀 **Mulai tanpa context**

👀 Demo Results

🏆 Waktunya Cek Hasil!

Apa yang AI kerjakan selama kita presentasi?

👉 Mari kita lihat bersama...

💡 Key Takeaways

1. 🤖 **AI = alat, bukan pengganti** — kamu tetap pilot
2. 📊 **Coverage & control** harus seimbang
3. 🧠 **Context > prompt** — termasuk audiens dan tujuan
4. 📋 **Struktur dulu** — audience → goal → flow → approach
5. 💎 **Kualitas tetap nomor satu** — review, test, verify
6. ⚙️ **System thinking** — konsistensi di seluruh produk

🙏 Terima Kasih!

🖥️ https://zainf.dev/ai-assisted-development

🔗 https://github.com/zainfathoni/zainf

Ada pertanyaan? ✋

Referensi

- Vibe Coding is Not the Same as AI-Assisted Engineering — Addy Osmani
- The 70% Problem — Addy Osmani
- Context Engineering — Addy Osmani
- How to Make Your Tech Stack AI-Friendly — Refactoring
- Kiro Specs — AWS
- Lenny's Newsletter, *How to build your PM second brain with ChatGPT*
- Lenny's Newsletter, *A guide to AI prototyping for product managers*